

Mudcard

- I am still not sure about how we get the most optimized prediction in logistic regression.
 - that's done by gradient descent
 - step 1: you start with an initial w
 - step 2: calculate what the logloss metric of your model is with the initial w given your X_{train} and y_{train}
 - step 3: calculate the derivative of the loss with respect to w
 - step 4: the new w is the initial w - learning rate * the loss derivative
 - repeat steps 2 to 4 until convergence is reached and you find the minimum of the loss

Lecture 8: Polynomial regression and regularization

By the end of this lecture, you will be able to

- Describe why regularization is important and what are the two types of regularization
- Describe how regularized linear regression works
- Describe how regularized logistic regression works

Regularization

By the end of this lecture, you will be able to

- Describe why regularization is important and what are the two types of regularization
- Describe how regularized linear regression works
- Describe how regularized logistic regression works

Polynomial regression

Let's work with a new example dataset

```
In [ ]: # load packages
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
import matplotlib
matplotlib.rcParams.update({'font.size': 11})

df = pd.read_csv('../data/regularization_example.csv')
X_ori = df['x0'].values.reshape(-1, 1)
y = df['y'].values
print(np.shape(X_ori))
print(np.shape(y))

# visualize the data
plt.figure(figsize=(5,3))
plt.scatter(X_ori,y)
plt.xlabel('feature')
plt.ylabel('target variable')
plt.show()
```

```
In [ ]: # lets generate more features because a linear model will obviously be insufficient to fit the data well
pf = PolynomialFeatures(degree = 40, include_bias=False)
X = pf.fit_transform(X_ori)
print(np.shape(X))
print(pf.get_feature_names_out())
```

We split data into train and validation!

```
In [ ]: from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=13)
print(np.shape(X_train), np.shape(y_train))
print(np.shape(X_val), np.shape(y_val))
```

Let's train and validate some linear regression models

Use the first feature only

```
In [ ]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# let's use only the first feature
linreg = LinearRegression(fit_intercept=True)
linreg.fit(X_train[:, :1], y_train)
print('intercept:', linreg.intercept_)
print('w:', linreg.coef_)

train_MSE = mean_squared_error(y_train, linreg.predict(X_train[:, :1]))
val_MSE = mean_squared_error(y_val, linreg.predict(X_val[:, :1]))
print('train MSE:', train_MSE)
print('val MSE:', val_MSE)

# let's visualize the model
x_model = np.linspace(0, 1, 100)
plt.figure(figsize=(5, 3))
plt.scatter(X_train[:, 0], y_train, color='k', label='training set')
plt.scatter(X_val[:, 0], y_val, color='r', label='validation set')
plt.plot(x_model, linreg.predict(x_model.reshape(-1, 1)), color='k', label='prediction')
plt.xlabel('feature')
plt.ylabel('target variable')
plt.legend()
plt.show()
```

Use all features

```
In [ ]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# use all features
linreg = LinearRegression(fit_intercept=True)
linreg.fit(X_train, y_train)
print('intercept:', linreg.intercept_)
print('ws:', linreg.coef_)

train_MSE = mean_squared_error(y_train, linreg.predict(X_train))
val_MSE = mean_squared_error(y_val, linreg.predict(X_val))
print('train MSE:', train_MSE)
print('val MSE:', val_MSE)

# let's visualize the model
x_model = np.linspace(0, 1, 1000)
plt.figure(figsize=(5, 3))
plt.scatter(X_train[:, 0], y_train, color='k', label='training set')
plt.scatter(X_val[:, 0], y_val, color='r', label='validation set')
plt.plot(x_model, linreg.predict(x_model.reshape(-1, 1))), color='k', label='prediction')
plt.ylim([-1, 1])
plt.xlabel('feature')
plt.ylabel('target variable')
plt.legend()
plt.show()
```

What to do?

- the model visibly performs poorly when only the original feature is used
- the model performs very good on the training set but poorly on the validation set when all features are used

- the ws are huge!

Regularization solves this problem!

Regularization

By the end of this lecture, you will be able to

- Describe why regularization is important and what are the two types of regularization
- **Describe how regularized linear regression works**
- Describe how regularized logistic regression works

Regularization to the rescue!

- let's change the cost function and add a **penalty term** for large ws
- **Lasso regression**: regularize using the l1 norm of w:

$$L(w) = \frac{1}{m} \sum_{i=1}^m [(w_0 + \sum_{j=1}^d w_j x_{ij} - y_i)^2] + \textcolor{red}{\alpha \sum_{j=0}^d |w_j|}$$

- **Ridge regression**: regularize using the square of the l2 norm of w:

$$L(w) = \frac{1}{m} \sum_{i=1}^m [(w_0 + \sum_{j=1}^d w_j x_{ij} - y_i)^2] + \textcolor{red}{\alpha \sum_{j=0}^d w_j^2}$$

- α is the regularization parameter (positive number), it describes how much we penalize large ws
- With the cost function changed, the derivatives in gradient descent need to be updated too!

Feature selection with Lasso regularization

- Least Absolute Shrinkage and Selection Operator
- cost = MSE + α * l1 norm of w

$$L(w) = \frac{1}{m} \sum_{i=1}^m [(w_0 + \sum_{j=1}^d w_j x_{ij} - y_i)^2] + \textcolor{red}{\alpha \sum_{j=0}^d |w_j|}$$
- ideal for feature selection
- as α increases, more and more feature weights are reduced to 0.

```
In [ ]: from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error

alpha = np.logspace(-7,0,29)
ws = []
models = []
train_MSE = np.zeros(len(alpha))
val_MSE = np.zeros(len(alpha))

# do the fit
for i in range(len(alpha)):
    # load the linear regression model
    lin_reg = Lasso(alpha=alpha[i])
    lin_reg.fit(X_train, y_train)
    ws.append(lin_reg.coef_)
    models.append(lin_reg)
    train_MSE[i] = mean_squared_error(y_train, lin_reg.predict(X_train))
    val_MSE[i] = mean_squared_error(y_val, lin_reg.predict(X_val))
```

```
In [ ]: plt.figure(figsize=(5,3))
plt.plot(alpha, ws)
plt.semilogx()
plt.xlabel('alpha')
plt.ylabel('ws')
plt.title('Lasso regression')
plt.tight_layout()
plt.savefig('./figures/lasso_coefs.png', dpi=300)
plt.show()
```

```
In [ ]: plt.figure(figsize=(5,3))
plt.plot(alpha,train_MSE,label='train score')
plt.plot(alpha,val_MSE,label='val score')
plt.semilogy()
plt.semilogx()
plt.xlabel('alpha')
plt.ylabel('MSE')
plt.legend()
plt.tight_layout()
plt.savefig('./figures/train_val_MSE_lasso.png',dpi=300)
plt.show()
```

Bias vs variance

- Bias: the model performs poorly on both the train and validation sets
 - high alpha in our example
- the model performs very well on the training set but it performs poorly on the validation set
 - low alpha in our example
 - lowering the alpha further would improve the train score but the validation score would increase
 - we don't do it because of convergence issues

The bias-variance trade off

- the curve of the validation score as a function of a hyper-parameter usually has a U shape if evaluation metric needs to be minimized, or an inverted U if the metric needs to be maximized
- choose the hyper-parameter value that gives you the best validation score

Quiz

Which alpha value gives the best validation score? Visualize the corresponding model!

```
In [ ]: # your code here
```

The bias-variance tradeoff with Ridge regularization

- $\text{cost} = \text{MSE} + \alpha * (\text{l2 norm of } w)^2$

$$L(w) = \frac{1}{m} \sum_{i=1}^m [(w_0 + \sum_{j=1}^d w_j x_{ij} - y_i)^2] + \alpha \sum_{j=0}^d w_j^2$$
- as α approaches 0, we reproduce the linear regression weights
- small α creates high variance
- large α creates high bias

```
In [ ]: from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

alpha = np.logspace(-10,0,51)

# arrays to save train and test MSE scores
train_MSE = np.zeros(len(alpha))
val_MSE = np.zeros(len(alpha))

ws = []
models = []
# do the fit
for i in range(len(alpha)):
    # load the linear regression model
    lin_reg = Ridge(alpha=alpha[i])
    lin_reg.fit(X_train, y_train)
    models.append(lin_reg)
    ws.append(lin_reg.coef_)
# train and test scores
```

```
train_MSE[i] = mean_squared_error(y_train, lin_reg.predict(X_train))
val_MSE[i] = mean_squared_error(y_val, lin_reg.predict(X_val))
```

```
In [ ]: plt.figure(figsize=(5,3))
plt.plot(alpha, ws)
plt.semilogx()
plt.ylim([-1e1, 1e1])
plt.xlabel('alpha')
plt.ylabel('ws')
plt.title('Ridge regression')
plt.tight_layout()
plt.savefig('./figures/ridge_coefs.png', dpi=300)
plt.show()
```

```
In [ ]: plt.figure(figsize=(5,3))
plt.plot(alpha, train_MSE, label='train score')
plt.plot(alpha, val_MSE, label='val score')
plt.semilogy()
plt.semilogx()
plt.xlabel('alpha')
plt.ylabel('MSE cost function')
plt.legend()
plt.tight_layout()
plt.savefig('./figures/train_val_MSE_ridge.png', dpi=300)
plt.show()
```

Quiz

Which α gives us the best tradeoff between bias and variance?

```
In [ ]: # your code here:
```

Regularization

By the end of this lecture, you will be able to

- Describe why regularization is important and what are the two types of regularization
- Describe how regularized linear regression works
- Describe how regularized logistic regression works**

Logistic regression

- Recap: the logloss metric is the cost function

$$L(w) = -\frac{1}{m} \sum_{i=1}^m [y_i \ln(y_i') + (1-y_i) \ln(1-y_i')] \\ L(w) = -\frac{1}{m} \sum_{i=1}^m [y_i \ln(\frac{1}{1+e^{-w_0 + \sum_{j=1}^d w_j x_{ij}}}) + (1-y_i) \ln(1-\frac{1}{1+e^{-w_0 + \sum_{j=1}^d w_j x_{ij}}})]$$

- the logloss metric with l1 regularization

$$L(w) = -\frac{1}{m} \sum_{i=1}^m [y_i \ln(y_i') + (1-y_i) \ln(1-y_i')] + \alpha \sum_{j=0}^d |w_j|$$

- the logloss metric with l2 regularization

$$L(w) = -\frac{1}{m} \sum_{i=1}^m [y_i \ln(y_i') + (1-y_i) \ln(1-y_i')] + \alpha \sum_{j=0}^d w_j^2$$

Logistic regression in sklearn

```
In [ ]: from sklearn.linear_model import LogisticRegression

log_reg_l1 = LogisticRegression(penalty='l1', C = 1/alpha) # C is the inverse of the regularization strength
log_reg_l2 = LogisticRegression(penalty='l2', C = 1/alpha)
# fit, predict, predict_proba are available
# log_reg.coef_ returns the w values

In [ ]: help(LogisticRegression)
```

Mudcard

```
In [ ]:
```